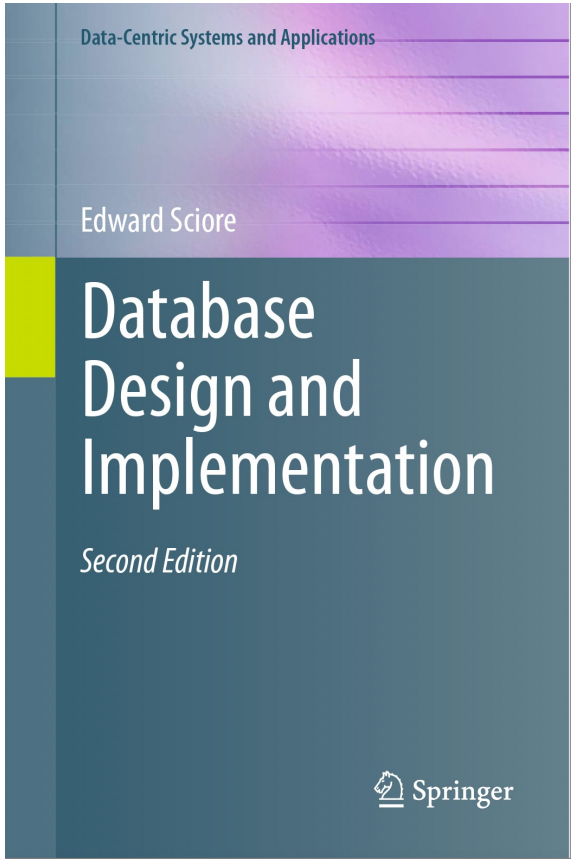
SimpleDB, parte 7

Luigi Bellomarini
luigi.bellomarini@uniroma3.it



SimpleDB, componenti per le transazioni

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

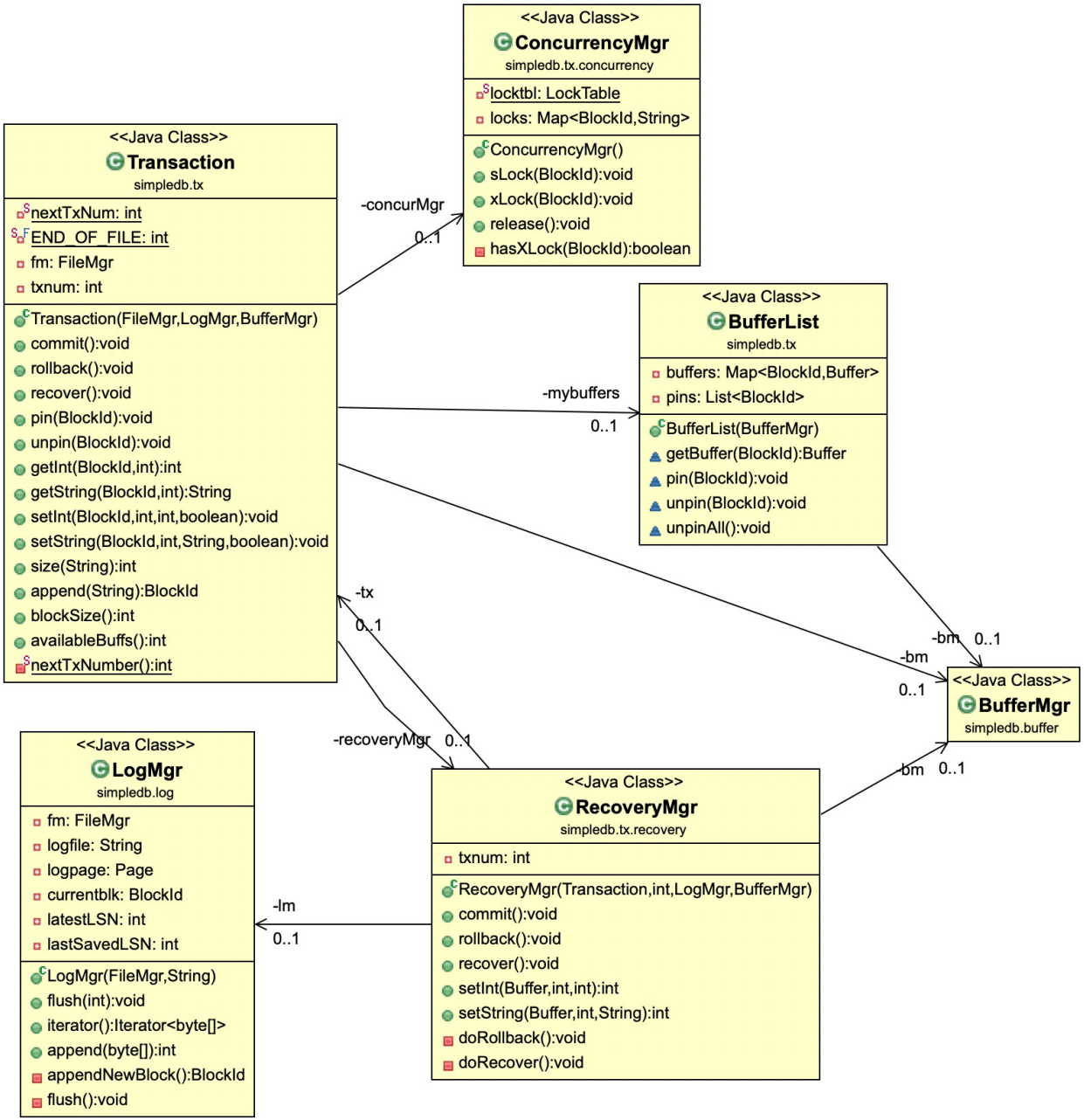
Record: gestisce i blocchi per i record delle tabelle

Concurrency Manager: gestisce la concorrenza

Buffer: mantiene in memoria le pagine per limitare gli accessi

Recovery Manager: garantisce l'affidabilità

File: legge e scrive le pagine su disco


<<Java Class>>
ConcurrencyMgr
simpledb.tx.concurrency
locktbl: LockTable
locks: Map<BlockId,String>
ConcurrencyMgr()
sLock(BlockId):void
xLock(BlockId):void
release():void
hasXLock(BlockId):boolean
<<Java Class>>
Transaction
simpledb.tx
nextTxNum: int
END_OF_FILE: int
fm: FileMgr
txnum: int
Transaction(FileMgr,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
pin(BlockId):void
unpin(BlockId):void
getInt(BlockId,int):int
getString(BlockId,int):String
setInt(BlockId,int,int,boolean):void
setString(BlockId,int,String,boolean):void
size(String):int
append(String):BlockId
blockSize():int
availableBuffs():int
nextTxNumber():int
-concurMgr
0..1
<<Java Class>>
BufferList
simpledb.tx
buffers: Map<BlockId,Buffer>
pins: List<BlockId>
BufferList(BufferMgr)
getBuffer(BlockId):Buffer
pin(BlockId):void
unpin(BlockId):void
unpinAll():void
-mybuffers
0..1
-tx
0..1
-bm
0..1
-bm
0..1
<<Java Class>>
BufferMgr
simpledb.buffer
-recoveryMgr
0..1
-bm
0..1
<<Java Class>>
LogMgr
simpledb.log
fm: FileMgr
logfile: String
logpage: Page
currentblk: BlockId
latestLSN: int
lastSavedLSN: int
LogMgr(FileMgr,String)
flush(int):void
iterator():Iterator<byte[]>
append(byte[]):int
appendNewBlock():BlockId
flush():void
-lm
0..1
<<Java Class>>
RecoveryMgr
simpledb.tx.recovery
txnum: int
RecoveryMgr(Transaction,int,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
setInt(Buffer,int,int):int
setString(Buffer,int,String):int
doRollback():void
doRecover():void

Example

<START , 0>

<SETINT, 0 , student.tbl, 0, 38, 2004, 2005>

<START , 1>

<START , 2>

<COMMIT, 1>

<SETSTRING, 2, junk, 44, 20, hello, ciao>

<START , 3>

<ROLLBACK, 3>

<SETSTRING, 0, student.tbl, 0, 46, amy, amiee>

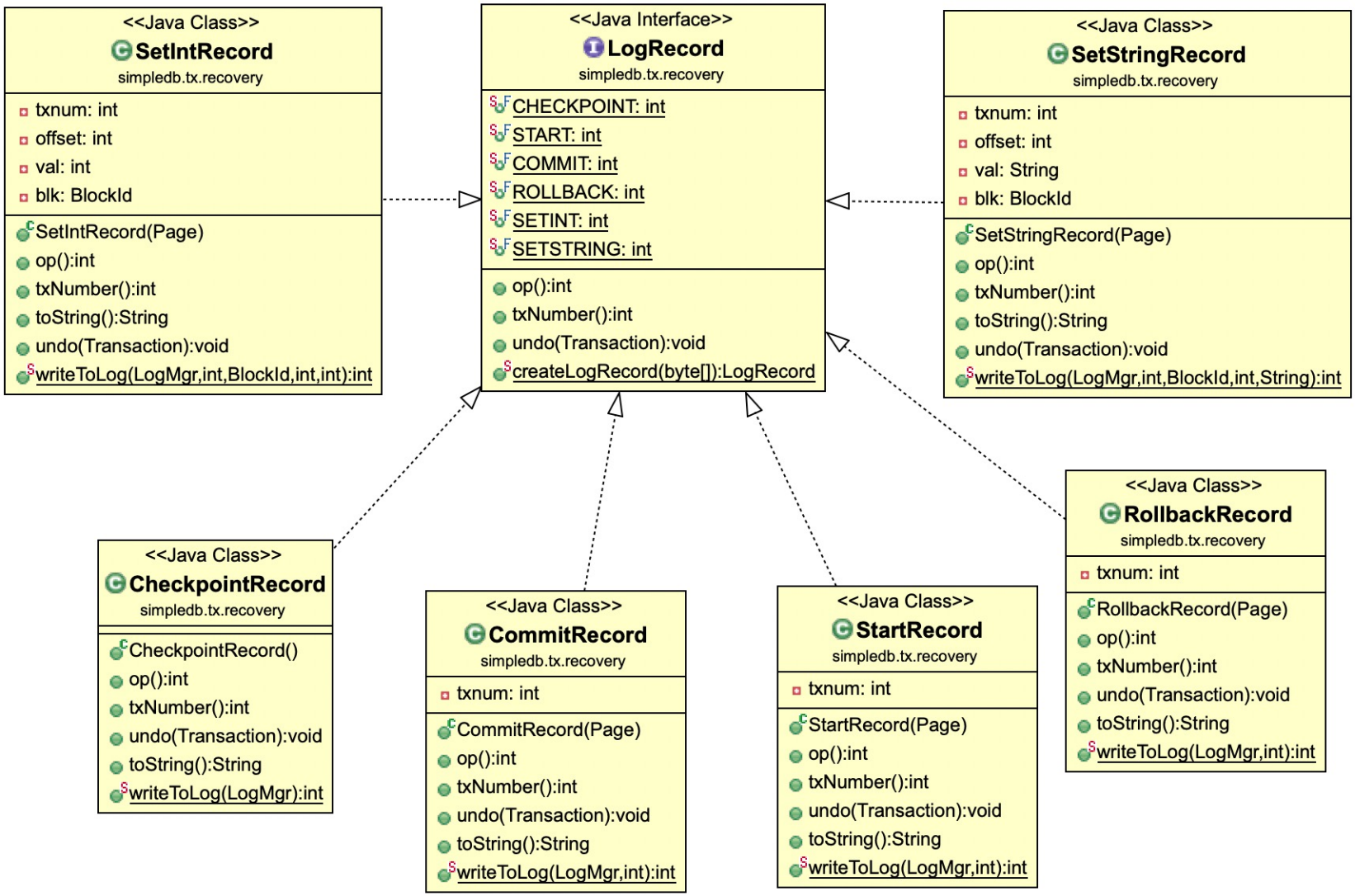
<COMMIT, 0>

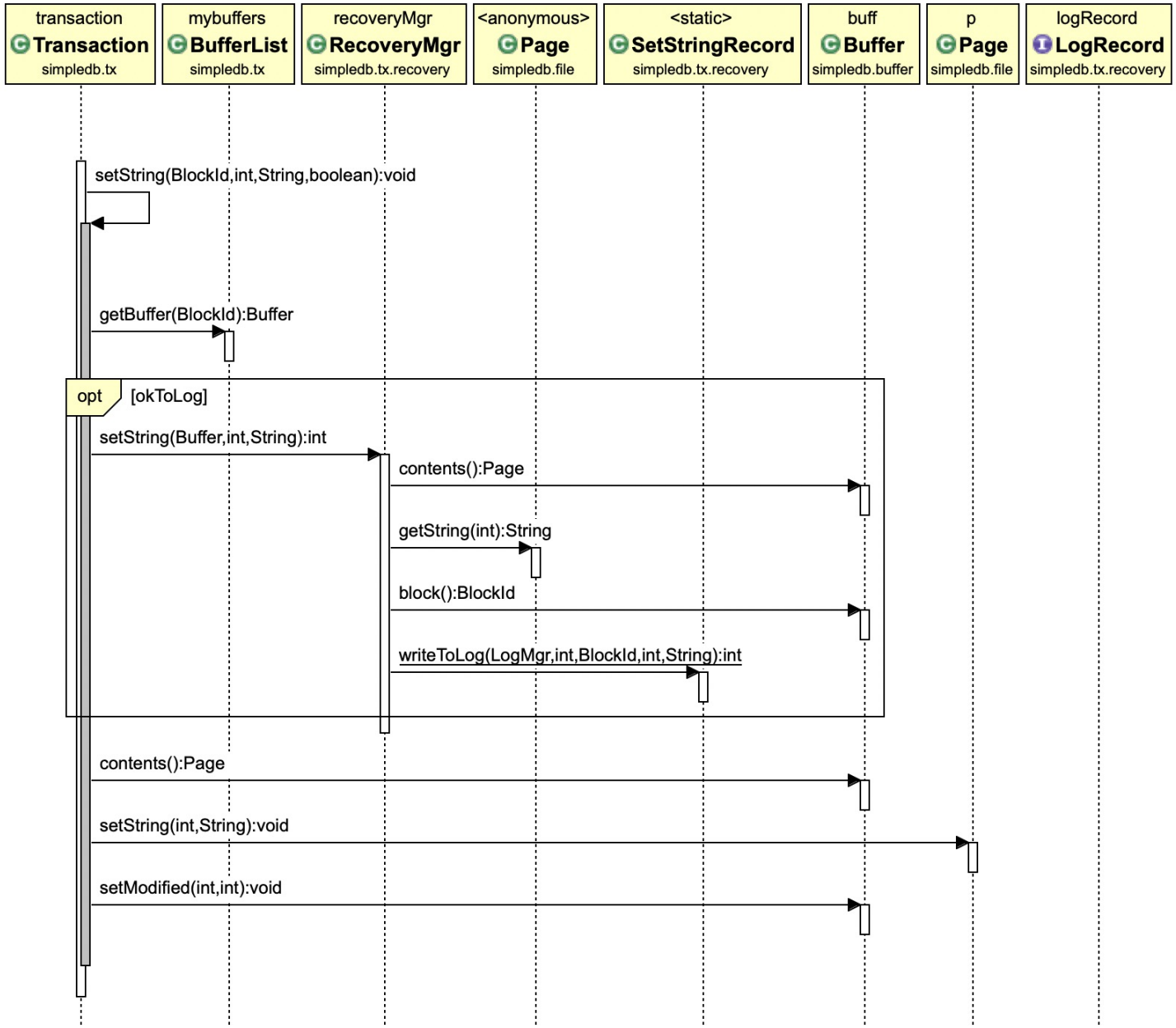
<SETINT, 2, junk, 66, 8, 0, 116>

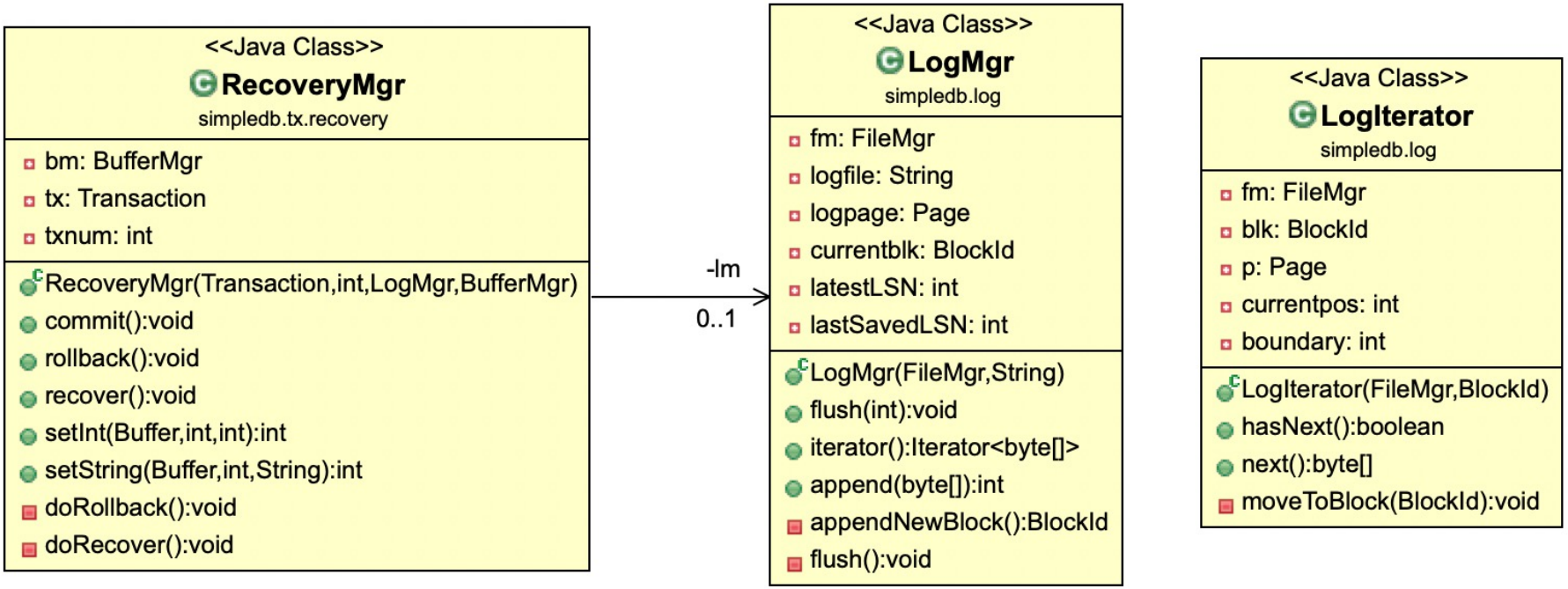
<COMMIT, 2>

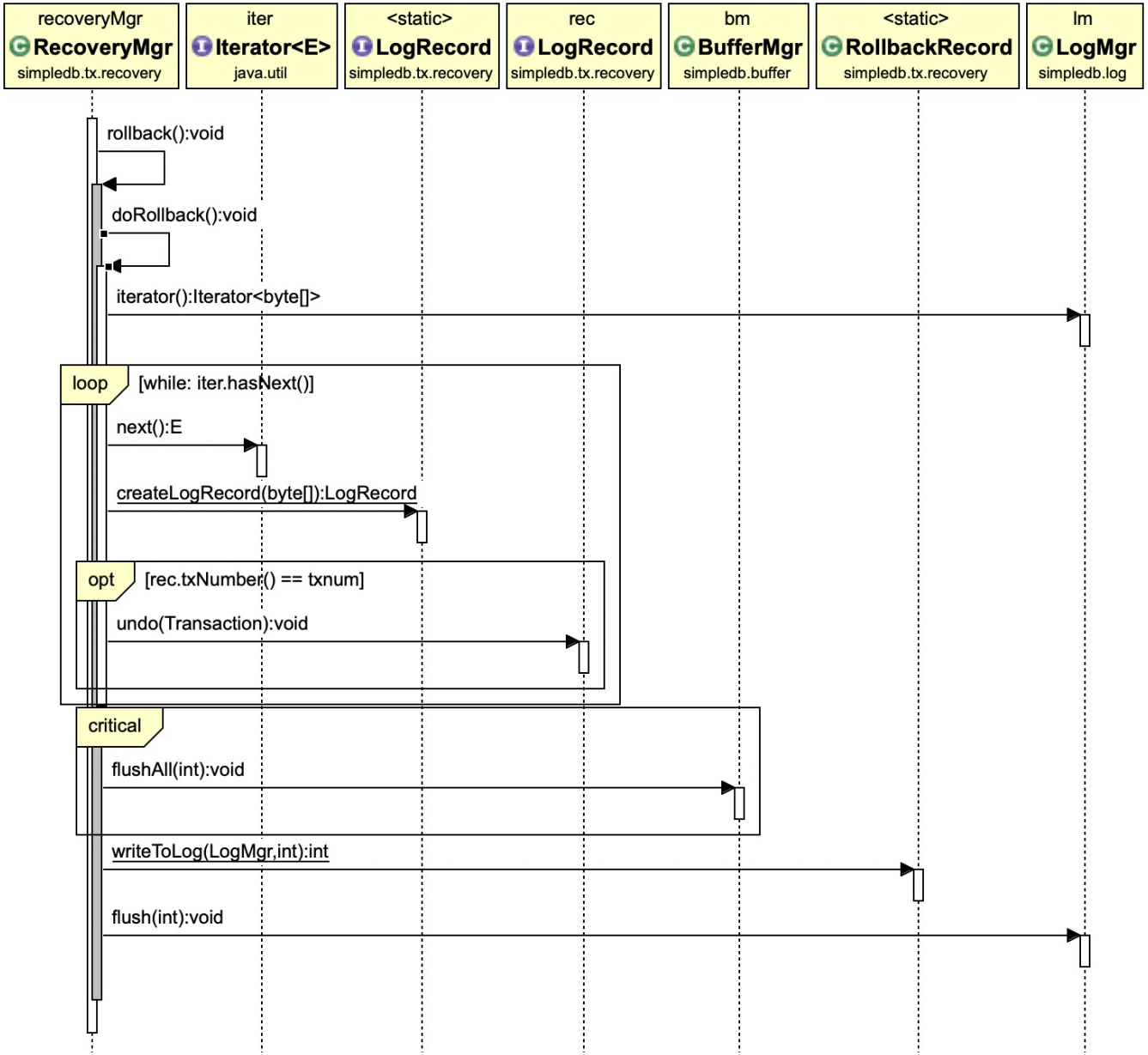
<START, 4>

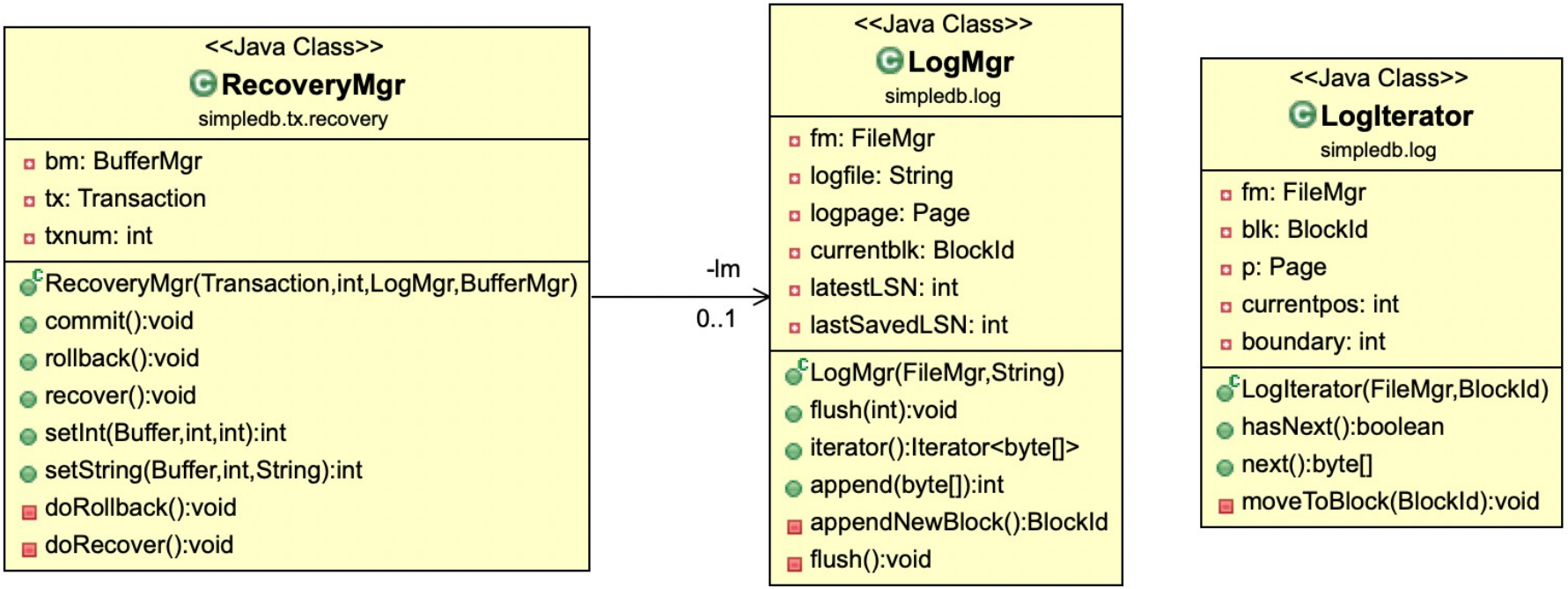
<SETINT, 4, junk, 33, 8, 543, 120>

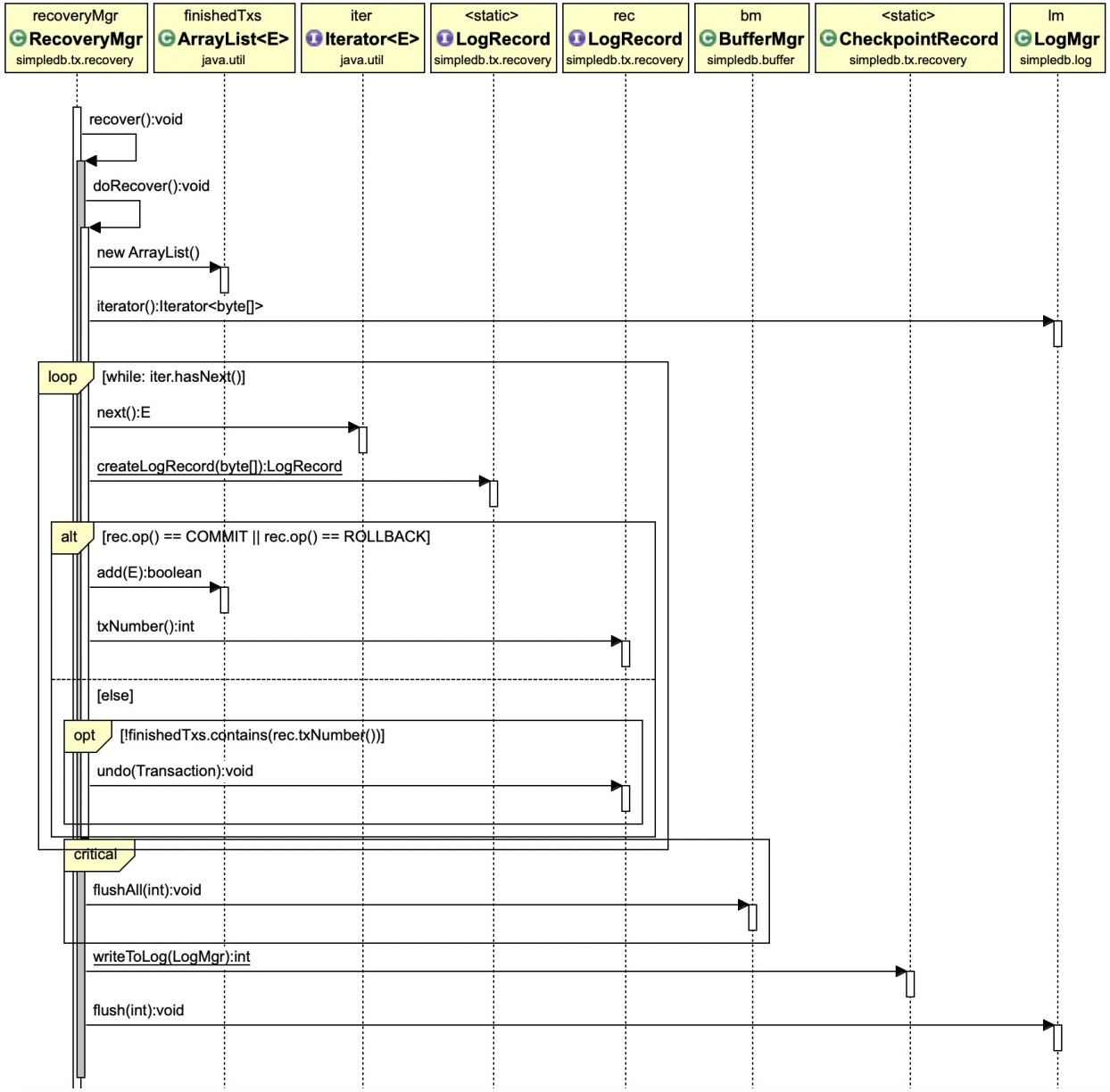

<<Java Class>>
SetIntRecord
simpledb.tx.recovery
txnum: int
offset: int
val: int
blk: BlockId
SetIntRecord(Page)
op():int
txNumber():int
toString():String
undo(Transaction):void
writeToLog(LogMgr,int,BlockId,int,int):int
<<Java Interface>>
LogRecord
simpledb.tx.recovery
CHECKPOINT: int
START: int
COMMIT: int
ROLLBACK: int
SETINT: int
SETSTRING: int
op():int
txNumber():int
undo(Transaction):void
createLogRecord(byte[]):LogRecord
<<Java Class>>
SetStringRecord
simpledb.tx.recovery
txnum: int
offset: int
val: String
blk: BlockId
SetStringRecord(Page)
op():int
txNumber():int
toString():String
undo(Transaction):void
writeToLog(LogMgr,int,BlockId,int,String):int
<<Java Class>>
CheckpointRecord
simpledb.tx.recovery
CheckpointRecord()
op():int
txNumber():int
undo(Transaction):void
toString():String
writeToLog(LogMgr):int
<<Java Class>>
CommitRecord
simpledb.tx.recovery
txnum: int
CommitRecord(Page)
op():int
txNumber():int
undo(Transaction):void
toString():String
writeToLog(LogMgr,int):int
<<Java Class>>
StartRecord
simpledb.tx.recovery
txnum: int
StartRecord(Page)
op():int
txNumber():int
undo(Transaction):void
toString():String
writeToLog(LogMgr,int):int
<<Java Class>>
RollbackRecord
simpledb.tx.recovery
txnum: int
RollbackRecord(Page)
op():int
txNumber():int
undo(Transaction):void
toString():String
writeToLog(LogMgr,int):int


transaction
Transaction
simpledb.tx
mybuffers
BufferList
simpledb.tx
recoveryMgr
RecoveryMgr
simpledb.tx.recovery
<anonymous>
Page
simpledb.file
<static>
SetStringRecord
simpledb.tx.recovery
buff
Buffer
simpledb.buffer
p
Page
simpledb.file
logRecord
LogRecord
simpledb.tx.recovery
setString(BlockId,int,String,boolean):void
getBuffer(BlockId):Buffer
opt
[okToLog]
setString(Buffer,int,String):int
contents():Page
getString(int):String
block():BlockId
writeToLog(LogMgr,int,BlockId,int,String):int
contents():Page
setString(int,String):void
setModified(int,int):void

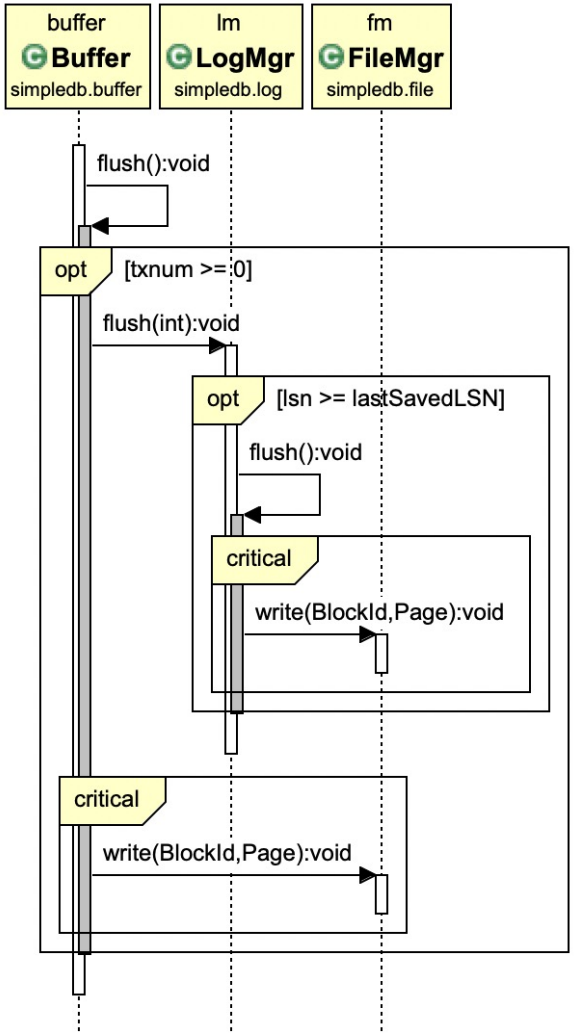
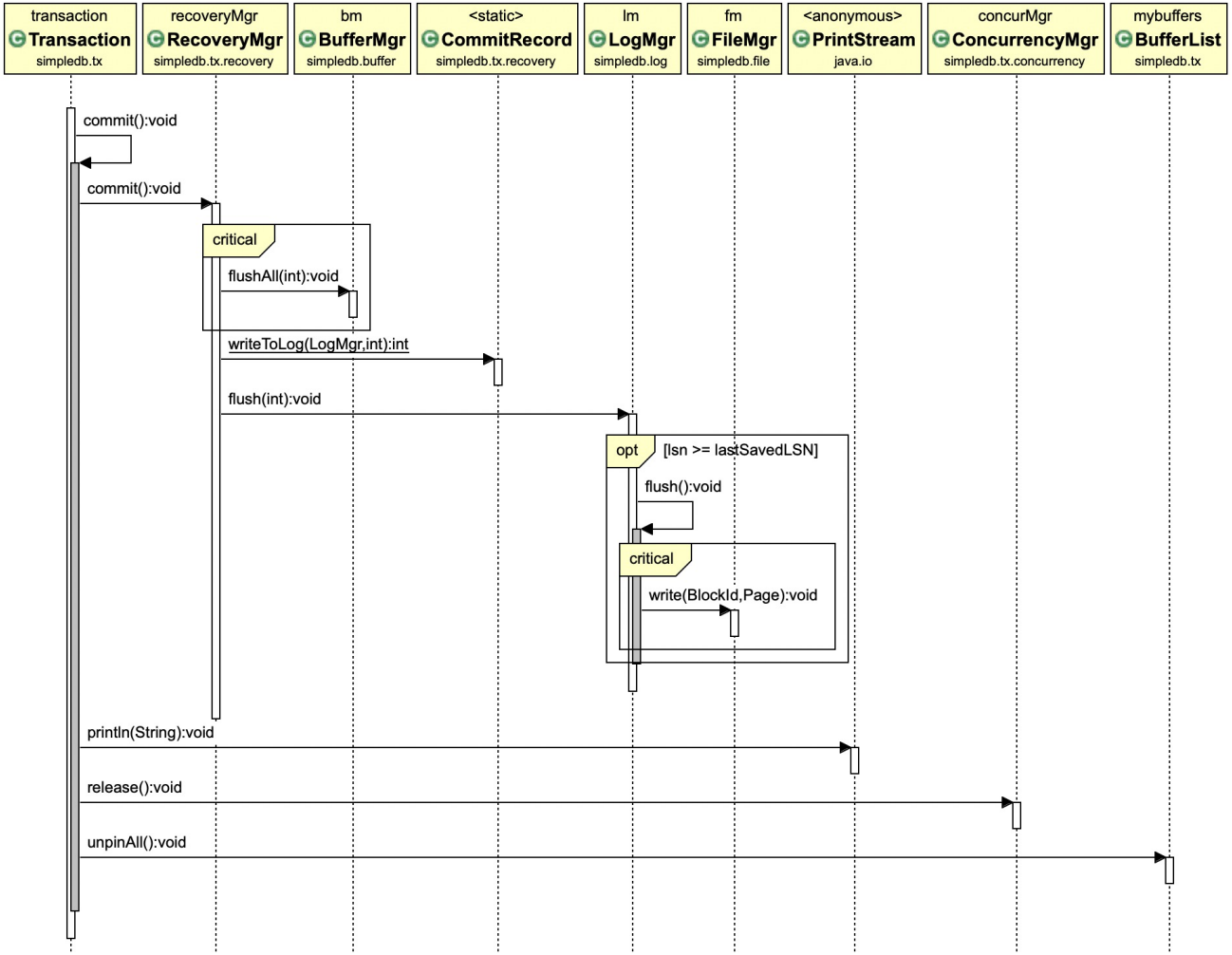

<<Java Class>>
RecoveryMgr
simpledb.tx.recovery
bm: BufferMgr
tx: Transaction
txnum: int
RecoveryMgr(Transaction,int,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
setInt(Buffer,int,int):int
setString(Buffer,int,String):int
doRollback():void
doRecover():void
-lm
0..1
<<Java Class>>
LogMgr
simpledb.log
fm: FileMgr
logfile: String
logpage: Page
currentblk: BlockId
latestLSN: int
lastSavedLSN: int
LogMgr(FileMgr,String)
flush(int):void
iterator():Iterator<byte[]>
append(byte[]):int
appendNewBlock():BlockId
flush():void
<<Java Class>>
LogIterator
simpledb.log
fm: FileMgr
blk: BlockId
p: Page
currentpos: int
boundary: int
LogIterator(FileMgr,BlockId)
hasNext():boolean
next():byte[]
moveToBlock(BlockId):void


recoveryMgr
RecoveryMgr
simpledb.tx.recovery
iter
Iterator<E>
java.util
<static>
LogRecord
simpledb.tx.recovery
rec
LogRecord
simpledb.tx.recovery
bm
BufferMgr
simpledb.buffer
<static>
RollbackRecord
simpledb.tx.recovery
lm
LogMgr
simpledb.log
rollback():void
doRollback():void
iterator():Iterator<byte[]>
loop
[while: iter.hasNext()]
next():E
createLogRecord(byte[]):LogRecord
opt
[rec.txNumber() == txnum]
undo(Transaction):void
critical
flushAll(int):void
writeToLog(LogMgr,int):int
flush(int):void


<<Java Class>>
RecoveryMgr
simpledb.tx.recovery
bm: BufferMgr
tx: Transaction
txnum: int
RecoveryMgr(Transaction,int,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
setInt(Buffer,int,int):int
setString(Buffer,int,String):int
doRollback():void
doRecover():void
-lm
0..1
<<Java Class>>
LogMgr
simpledb.log
fm: FileMgr
logfile: String
logpage: Page
currentblk: BlockId
latestLSN: int
lastSavedLSN: int
LogMgr(FileMgr,String)
flush(int):void
iterator():Iterator<byte[]>
append(byte[]):int
appendNewBlock():BlockId
flush():void
<<Java Class>>
LogIterator
simpledb.log
fm: FileMgr
blk: BlockId
p: Page
currentpos: int
boundary: int
LogIterator(FileMgr,BlockId)
hasNext():boolean
next():byte[]
moveToBlock(BlockId):void


recoveryMgr
RecoveryMgr
simpledb.tx.recovery
finishedTxs
ArrayList<E>
java.util
iter
Iterator<E>
java.util
<static>
LogRecord
simpledb.tx.recovery
rec
LogRecord
simpledb.tx.recovery
bm
BufferMgr
simpledb.buffer
<static>
CheckpointRecord
simpledb.tx.recovery
lm
LogMgr
simpledb.log
recover():void
doRecover():void
new ArrayList()
iterator():Iterator<byte[]>
loop
[while: iter.hasNext()]
next():E
createLogRecord(byte[]):LogRecord
alt
[rec.op() == COMMIT || rec.op() == ROLLBACK]
add(E):boolean
txNumber():int
[else]
opt
[!finishedTxs.contains(rec.txNumber())]
undo(Transaction):void
critical
flushAll(int):void
writeToLog(LogMgr):int
flush(int):void

Chi effettua il flush del log (WAL)?



Commit precedenza in SimpleDB